

CHANNEL Z0 – ARCHIVE INTAKE (STATION BULLETIN Nº 4)

DOC NO. RL-Z0-ARCHIVE-FETCH-A REV. A RIPOSTE LABORATORIES

CONTENTS

▶ The one rule	2
▶ Quick start	2
▶ The recipes	4
▶ How it works, and what was verified	6
▶ Picking up where this left off	7
State of play, 2026-08-06	8
Known gaps, honestly	9
▶ When a viewer flags something	9
▶ Testing without the bytes	11
▶ Editing the recipes	12

CHANNEL Z0 · CH 0 · DESIG RL-Z0
ARCHIVE INTAKE – stocking the library from archive.org
public domain only, verified on the way in

`docs/programming.md` `RL-Z0-PROGRAMMING` defines a full broadcast week and splits it

into two halves: **the ours** (Ground Zero, Lab Hour, the Desk, the sign-off) and **the acquired** (Prelinger Theatre, the Cartoon Block, the themed feature nights). This bulletin is about the second half – how `tools/fetch-archive.sh` fills those folders from the Internet Archive, and how to pick the job up where it was left.

SEC.01

THE ONE RULE

Only items carrying a public-domain licence mark are ever downloaded.

This is not decoration. archive.org is a library, not a rights clearinghouse, and some of its collections are a minefield. The obvious candidate for the ad break – `collection:(classic_tv_commercials)`, 7,985 items – contains, among other things, a **2007 FedEx Kinko's television commercial**. Of those 7,985 items only **421** carry a public-domain licence.

So every recipe in the script appends `licenseurl:(*publicdomain*)`, and the matched licence URL is written into the manifest for each file, so any item on the channel can be traced back to the mark it was aired under. This keeps the station where `docs/ad-standards.md` RL-Z0-AD-STANDARDS wants it: **air only what's yours to air.**

A licence mark is an uploader's claim, not a legal opinion. It is the best signal the platform offers and it is what this tool trusts. Spot-check anything you plan to put in a tentpole slot.

SEC.02

QUICK START

BASH

```
tools/make-media-tree.sh      # creates the destinations (idempotent)
tools/fetch-archive.sh --list # the recipes
tools/fetch-archive.sh --dry-run --all
tools/fetch-archive.sh --all  # go
tools/fetch-archive.sh --status # how full is the library
```

Requires `curl`, `jq`, and `ffprobe` (ffmpeg). Nothing else – no `ia` CLI, no API key, no account.

Useful knobs, all environment variables:

VARIABLE	DEFAULT	WHAT
Z0_MEDIA_ROOT	/media/ channelz0	Where the library lives
Z0_RATE_LIMIT	(none)	e.g. 2M – caps download rate so intake doesn't fight the uplink
Z0_FETCH_SLEEP	2	Seconds between items; be a good guest
Z0_MAX_MB	3000	Skip any single file bigger than this

Run intake during the overnight colour-bar block. The playout PC is sharing a residential line with the thing that is actually on air; a feature is a few hundred megabytes and `-all` is tens of gigabytes. `Z0_RATE_LIMIT=2M` and a sign-off-to-sunrise window is the polite shape.

SEC.03

THE RECIPES

Each slot is `destination`, a duration window, a default count, and a query. The duration window is what separates a filler PSA from a feature and is a first-class part of the recipe, not a post-filter.

SLOT	→ DESTINATION	DURATION	ERSATZTV COLLECTION	SOURCE
psas	psas/	20-300 s	Vintage PSAs (ad-break filler)	Prelinger
prelinger	prelinger/	5-40 min	Prelinger (Prelinger Theatre)	Prelinger
cartoons	cartoons/	1-20 min	Cartoons (Cartoon Block)	animationandcartoons
slowtv	slowtv/	15 min-3 h	Slow TV (Stretch & Coffee, Night Pattern)	subject + year bound
noir	movies/noir/	40 min-3 h	Features · Noir	feature_films + subject
scifi	movies/scifi/	40 min-3 h	Features · SciFi	feature_films + subject
docs	movies/docs/	40 min-3 h	Features · Docs	feature_films + subject
serials	movies/ serials/	10-60 min	Features · Serials	subject + year bound
classics	movies/ classics/	40 min-3 h	Features · Classics	feature_films + subject
cult	movies/cult/	40 min-3 h	Features · Cult	feature_films + subject

Pool sizes measured 2026-08-06, public-domain-marked only:

```
prelinger 1875    animationandcartoons 719
noir 137   scifi 424   docs 45   serials 12   classics 1210   cult 36
```

cult (36) is **thin** – the script will report **pool exhausted** rather than silently under-delivering. Widen that query if the themed night needs more; see **Editing the recipes** below.

serials used to be thin for a different reason: the recipe asked **collection:(feature_films)** for **subject:(serial)**, and the whole of archive.org only answers that with **12** items, which is why SERIAL NIGHT could never be stocked more than five chapters deep. Dropping the collection filter and accepting **serials/cliffhanger/chapter play** as well takes the pool to **404**.

On the year bound. **serials** and **slowtv** both end in **year:[1900 TO 1985]**. The public-domain licence mark is also worn by present-day CC0 uploads, so an undated subject query returns 2020s explainer videos next to the mid-century material – legally fine, tonally wrong for a channel that is pretending to be older than streaming. The other slots don't need it: their collections are already vintage by construction.

One block in **programming.md** is still deliberately **not** fetched: **Music Loops** wants rights-owned music, and there is no honest query for that.

slowtv **is** fetched, but read this before trusting it. What Stretch & Coffee and Sunday Service actually want is **your own** long takes of the neighbourhood, and no query returns those. The recipe fetches vintage scenic, travelogue and railroad film instead – the same texture, the same era, and public-domain-marked, but it is a **stand-in**, not the real thing. Replace it with your own footage when you have some. **bumpers/** is not fetched either – those come from **make-ident.sh**.

SEC.04

HOW IT WORKS, AND WHAT WAS VERIFIED

Everything below was checked against the live API on **2026-08-06** before being written into the script.

1 • Discovery – `advancedsearch.php` with `sort[]=random`. The important detail. Walking results in identifier order gives you a library where every title starts with "A", because these collections are large and we only ever take the first few dozen. `sort[]=random` samples the whole corpus – a 300-row page spreads properly across the alphabet. It is stable between calls (so resuming sees the same order) but is **not** a stable total order across pages: **pages overlap by roughly a sixth**, so candidates are deduped by identifier, preserving first-seen order. Window limit is 10,000 results (page × rows), far past what a station needs.

The newer `services/search/v1/scrape` endpoint has cursor paging and no window limit, but **rejects count below 100** and offers no random sort – which is exactly the ordering problem above. It was tried and rejected for this job.

2 • Selection – `/metadata/<identifier>`. Returns every file in the item. The script picks the first available of:

h.264 → 512Kb MPEG4 → MPEG4 → HiRes MPEG4 → Ogg Video

h.264 first is deliberate: it is the web-sized derivative. The MPEG4 entry is often the preservation master – one twelve-minute Prelinger short lists a **723 MB .m4v** next to an **80 MB .mp4** of the same film. `Z0_MAX_MB` catches the rest.

Durations come from the file's `length` field, which archive.org reports either as seconds (768.54) or as a clock (12:48, 1:02:33). Both are parsed.

3 • Download – `https://archive.org/download/<id>/<url-encoded filename>`. File-names contain spaces and parentheses, so they are @uri-encoded through jq. Downloads land in `<name>.part` and are `curl -C -` resumable; a killed run continues mid-file rather than starting over.

4 • **Verification – ffmpeg**. Nothing is accepted into the library until ffmpeg can read a duration from it. A file that won't probe is deleted and logged **failed-probe**. An unplayable file is a black hole in the broadcast day, and the schedule is unattended by design.

5 • **Filing**. **Title (Year) [identifier].ext**. Close enough to Plex naming that ErsatzTV's scanner files features correctly – it ignores the trailing bracket – and the identifier keeps every file traceable to its archive.org item.

SEC.05

PICKING UP WHERE THIS LEFT OFF

State lives with the media, not in git (**media/** is gitignored, and the library belongs to the playout PC):

```
$Z0_MEDIA_ROOT/.z0-archive/
├─ manifest.jsonl          one line per item ever attempted – the source of truth
└─ candidates/<slot>.jsonl  cached search results per slot
```

manifest.jsonl is append-only JSONL. One line per attempt:

JSON

```
{ "slot": "psas", "identifier": "tupperware_2", "title": "Tupperware Commercial #2",
  "year": "", "file": "/media/channelz0/psas/Tupperware Commercial #2 [tupperware_2].mp4",
  "format": "h.264", "bytes": 8303733, "duration": 80,
  "licenseurl": "http://creativecommons.org/licenses/publicdomain/", "status": "ok" }
```

status is **ok**, **failed-download**, or **failed-probe**. Any **identifier** already in the **manifest** is **skipped on the next run**, successes and failures alike – so a re-run always makes forward progress and never re-fetches. That is the entire resume mechanism; it is deliberately boring.

To inspect:

BASH

```

tools/fetch-archive.sh --status                # per-slot counts + hours on disk
jq -r 'select(.status="ok")|.file' .z0-archive/manifest.jsonl
jq -r 'select(.status|startswith("failed"))|.identifier' .z0-archive/manifest.jsonl
jq -rs 'group_by(.licenseurl)|map({(. [0].licenseurl):length})|add' .z0-archive/manifest.jsonl

```

To **retry** failures, delete those lines from the manifest and re-run. To **re-roll** a slot's candidate pool (different films, same query), delete `candidates/<slot>.jsonl`. To **replace** an item you don't want on air, delete the file **and** its manifest line.

`-status` reports hours-per-slot. Aim at what the grid actually consumes: the Cartoon Block and Prelinger Theatre run daily, so a few hours each is thin – treat the default counts as a first pass and raise `-count` on the slots the schedule leans on.

STATE OF PLAY, 2026-08-06

The script is finished and verified end to end. **No library has been stocked** – all runs so far were against a scratch media root, not a real playout PC. What was proven, with live downloads:

- ▶ `psas` fetched 3 real files (Rohm & Haas Plexiglas 1947, a Crest ad, a Tupperware ad), all `ffprobe-clean` at 279 s / 41 s / 80 s, manifest written correctly.
- ▶ Resume verified: a second run skipped all three and offered three different items.
- ▶ `noir` dry run surfaced **Detour** (1945), **The Second Woman** (1951), **The Scar** – genuine public-domain noir, correctly duration-filtered.
- ▶ `serials` correctly reported `pool exhausted` at 6 of 20 requested.

The next operator should run `tools/make-media-tree.sh` then `tools/fetch-archive.sh --all` on the actual playout PC, overnight, with `Z0_RATE_LIMIT` set.

KNOWN GAPS, HONESTLY

- ▶ **Thin pools.** `serials` (12) and `cult` (36) can't fill a themed night on their own. Either widen the subject terms or accept heavy repetition.
- ▶ **Subject tags are uneven.** `archive.org` subjects are uploader-supplied, so the themed queries are approximate – a noir may land in `classics`. Re-file by hand; the schedule won't notice, but you will.
- ▶ **No transcode on intake.** Files arrive at whatever resolution the derivative is; ErsatzTV's Z0 Broadcast profile normalises at play-out. Loudness is **not** normalised – `normalize-ad.sh` exists for submitted spots, and vintage material varies wildly. If the Cartoon Block is noticeably louder than the feature, that is why.
- ▶ **Nothing fills `slowtv/`, `bumpers/`, or `commercials/local`.** By design.
- ▶ **No dedupe by title.** Two uploads of the same film have different identifiers and both land (the `serials` dry run pulled both `Tih-Minh` and `Tin-Minh-1918`).

SEC.06

WHEN A VIEWER FLAGS SOMETHING

The intake policy is public. `site/index.html` carries it as **SEC.05, the Rights Desk**: where the acquired half of the schedule came from, that only public-domain-marked items are ever fetched, and – in as many words – that a mark is an uploader's claim rather than a ruling. Under it sits a flag link, a `mailto:`: the page fills in on the viewer's behalf, because a viewer cannot name what they are looking at. The picture carries no identifier; the page knows one.

A claim arrives with these lines above whatever the sender wrote:

```
PROGRAMME:  THE VANCOUVER REEL
TITLE FROM:  the tower, live
SLOT:       SUNDAY 14:00 · THE VANCOUVER REEL
FLAGGED:    2026-08-24 14:07:31 UTC-07:00
FLAGGED UTC: 2026-08-24T21:07:31.412Z
WATCHING:   yes
```

Read **TITLE FROM** first. **The tower, live** is the title master control actually set,

which names a programme. **Today's schedule** means the page could not reach Owncast and fell back to its own copy of the grid – the claim then points at a **slot**, not a file, and the two timestamps are what narrow it down. Both are always sent: local time carries its UTC offset because a claim is read in another timezone weeks later, and a bare wall clock is not a moment.

To act on one:

1. **Find the item.** `PROGRAMME` matches the on-air title; the file is `Title (Year) [identifier].ext` and the identifier is the archive.org item. `bash jq -r 'select(.title|test("vancouver";"i"))' .z0-archive/manifest.jsonl`
2. **Read its manifest line.** `licenseurl` is the mark it was aired under – that, not a memory of the query, is what the station acted on.
3. **Take it off air, then check.** Delete the file, and **keep** the manifest line: `already_have()` skips any identifier that appears in the manifest at all, whatever its status, so the line is what stops the next `-all` from fetching it straight back. Set its `status` to `withdrawn` while you are in there; nothing reads the value, and the next operator will read it. (The **replace** recipe above deletes both, on purpose – that is for an item you merely didn't want, and which may be fetched again.)
4. **Say so at the source** if the mark itself was wrong. The next station to run this script reads the same field off the same item.

Claims land in the same mailbox as spot submissions; the subject line (`Channel Z0 – rights claim – ...`) is what separates them.

SEC.07

TESTING WITHOUT THE BYTES

A stocked library is tens of gigabytes and one feature is a few hundred megabytes, which makes **"does my schedule actually work"** an expensive question – especially on a playout PC that is also the thing on air.

`tools/make-proxies.sh` mirrors the library into stand-ins: same tree, same file-names, **same durations**, a fraction of a percent of the bytes.

BASH

```
tools/make-proxies.sh          # low-res transcodes of everything
tools/make-proxies.sh --placeholder # labelled cards instead of video
tools/make-proxies.sh psas/ movies/noir/ # just these
tools/make-proxies.sh --dry-run
```

Output goes to `$Z0_MEDIA_ROOT-test` (override with `-out` or `Z0_PROXY_ROOT`). Point a throwaway ErsatzTV channel at it and the broadcast day behaves exactly as it will on air – it just looks like a webcam from 1998.

Duration is the whole point. Schedules, pad-to-the-half-hour, and the protected 19:00 anchor are arithmetic on runtimes, so proxies keep runtimes exact. `-seconds N` trims for a fast smoke test and deliberately breaks that property; the script says so when you use it.

MODE	WHAT YOU GET	MEASURED
transcode (default)	320px, crf 40, mono audio – the actual content	6.9% of source bytes
-placeholder	a Z0 card naming the title and the runtime it stands in for	0.6% of source bytes

Useful options: `-width`, `-crf`, `-jobs N` (default 4), `-force`, `-dry-run`. Re-runs skip anything already built, so it resumes like the fetcher does.

Two things worth knowing if you touch the placeholder path:

- ▶ **The cost is the audio, not the video.** A static frame is nearly free to x264; AAC-encoding 90 minutes of **silence** at 48k stereo measured 38 s against 6 s for the video. The silence is 22050 mono for that reason, which takes a feature-length card from ~43 s to ~24 s. Framerate barely matters.
- ▶ **The label is never escaped, it is handed over out-of-band.** A runtime like 1:30:00 reads as the next filter option and kills the filtergraph, and a title like Kelowna's Own closes drawtext's quote early and drops every drawtext after it. This used to be handled by flattening the title to a safe charset, which is worse than it sounds: the card rendered fine, exit 0, right duration, and quietly said KELOWNAS OWN – and the label is the only thing a placeholder is **for**. Both the title and the runtime now go through z0_text from z0-lib.sh, which writes the words to a file and points drawtext at it with textfile=...:expansion=none. There is no escaping layer left to get wrong. Only newlines and tabs are still stripped, because those break the card's **layout**, not its parsing.

SEC.08

EDITING THE RECIPES

The RECIPES array near the top of the script is the whole configuration:

```
| slot | destination | min secs | max secs | default count | query
```

Add a themed night by adding a line – a Features · Westerns slot is westerns|movies/westerns|2400|10800|10|collection:(feature_films) AND mediatype:(movies) AND licenseurl:(*publicdomain*) AND subject:(western). Keep licenseurl:(*publicdomain*) in any query you add. Check the pool size first:

BASH

```
| curl -s -G 'https://archive.org/advancedsearch.php' \
  --data-urlencode 'q=<your query>' --data-urlencode 'rows=0' \
  --data-urlencode 'output=json' | jq '.response.numFound'
```

Intake runs overnight, between the sign-off and the sunrise. The bars don't mind.

SOURCE docs/archive-fetch.md SHA 6cf0eda10d89 BUILT 2026-08-24